

Pawdentify – Deployment & Infrastructure Documentation

1. Executive Summary

Pawdentify is a full-stack dog breed classification platform. This document outlines the production-grade deployment strategy, utilizing a hybrid cloud architecture with AWS for backend compute/storage and Vercel for frontend hosting. The system is secured via SSL/TLS encryption and follows the principle of least privilege for cloud resource management.

2. System Architecture

The application architecture is divided into four primary layers:

- **Frontend:** Developed with React and Tailwind CSS; hosted on **Vercel** with automated HTTPS.
- **Backend:** A FastAPI application hosted on an **AWS EC2** instance (Ubuntu 22.04) utilizing **Nginx** as a reverse proxy.
- **Model Storage:** A trained TensorFlow model (.keras) stored in **AWS S3** and fetched dynamically.
- **Database:** **MongoDB Atlas** for managed, cloud-native data persistence.

3. Model Management (AWS S3)

To decouple heavy model files from the application code, the trained model is stored externally.

- **Storage:** The model is hosted in a private AWS S3 bucket.
- **Security:** Access is managed via an IAM user configured with the `AmazonS3ReadOnlyAccess` policy.
- **Integration:** The backend uses the `boto3` library to download the model to the local instance storage during the initial startup sequence.

Required Environment Variables:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`
- `S3_BUCKET`
- `S3_MODEL_KEY`

- `LOCAL_MODEL_PATH`

4. Backend Infrastructure (AWS EC2)

The backend services are hosted on an Ubuntu 22.04 LTS instance.

Deployment Workflow:

1. **Environment Setup:** Installed Python 3.10+, pip, and Git on the EC2 instance.
2. **Code Deployment:** Cloned the repository and established a Python virtual environment to isolate dependencies.
3. **Dependency Management:** Installed required packages via `requirements.txt`.
4. **Configuration:** Created a localized `.env` file on the server to store sensitive credentials (API keys, DB URIs) securely.
5. **Process Management:** Utilized `tmux` to ensure the FastAPI server (running via Uvicorn) remains active after SSH session termination.

5. Networking & Security (Nginx & HTTPS)

To ensure secure data transmission and professional routing, the following networking stack was implemented:

- **DNS Configuration:** A custom domain was mapped to the EC2 public IP using DuckDNS (pawdentify-backend.duckdns.org).
- **Reverse Proxy:** Nginx was configured to sit in front of the Uvicorn server. This handles incoming requests on Port 80/443 and forwards them to the FastAPI application.
- **SSL/TLS Encryption:** Certbot (Let's Encrypt) was used to provision and auto-renew SSL certificates, enabling full HTTPS support.

Production Backend URL: <https://pawdentify-backend.duckdns.org>

6. Frontend Deployment (Vercel)

The React-based frontend is deployed via Vercel's edge network for low-latency delivery.

Configuration Steps:

1. Connected the GitHub repository to the Vercel platform for Continuous Deployment (CD).
2. Configured environment variables within the Vercel dashboard to point the frontend to the production backend:
 - `VITE_API_URL` : The HTTPS DuckDNS URL.
 - `VITE_CLERK_PUBLISHABLE_KEY` : For user authentication.
 - `VITE_MAPPLS_MAP_SDK_KEY` : For location-based features.

7. Security & Cross-Origin Resource Sharing (CORS)

To prevent unauthorized access, strict security protocols were implemented:

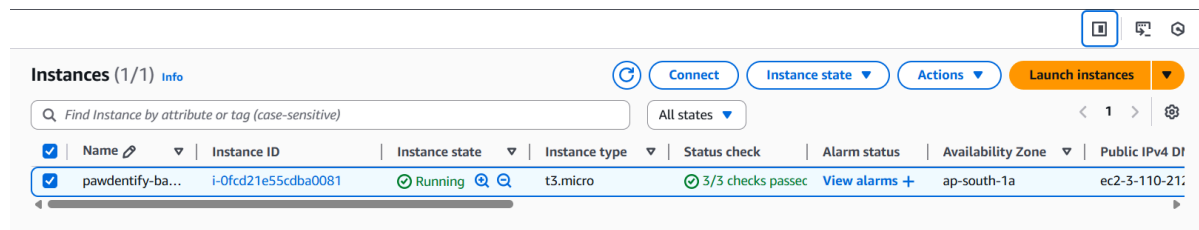
- **CORS Policy:** The FastAPI backend explicitly allows only the Vercel production domain and local development environments to make API requests.
- **Secret Management:** No sensitive keys or credentials are committed to version control. All secrets are managed via server-side environment variables.
- **IAM Policies:** AWS access is restricted to the minimum permissions required for S3 retrieval.

8. Final Production Access

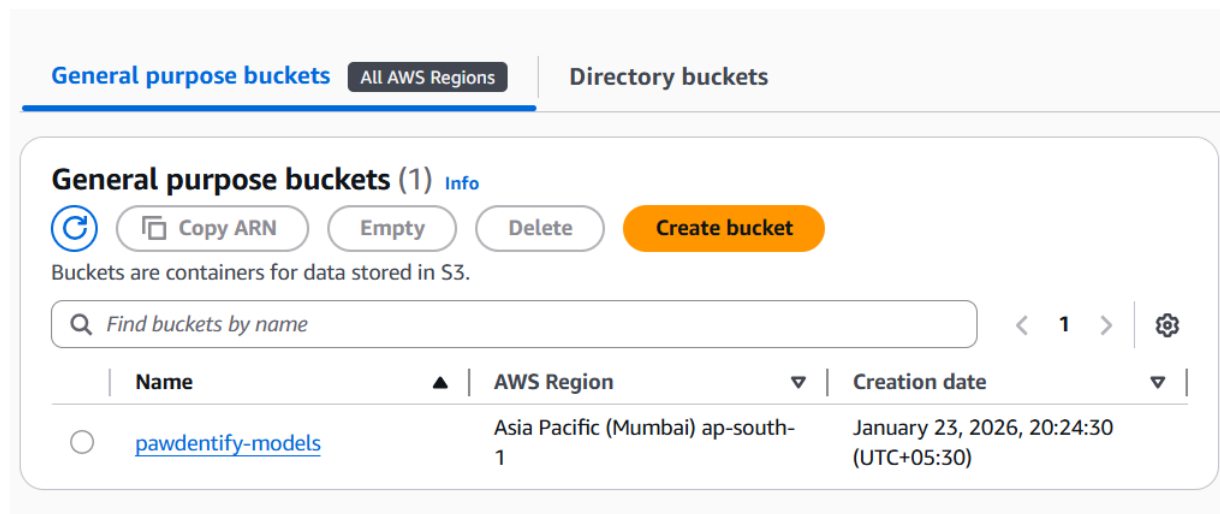
- **Frontend Application:** <https://pawdentify-frontend.vercel.app>
- **Backend API:** <https://pawdentify-backend.duckdns.org>

9. Screenshots

Instance :



S3 :



MongoDB Cluster :

Data Explorer

- My Queries
- Data Modeling

CLUSTERS (1)

Search clusters

- powdentify
 - admin
 - local
 - powdentify
 - breed_searches
 - pets
 - search_history
 - user_settings

breed_searches powdentify +

powdentify > powdentify

View monitoring
Visualize your data
Create collection
Refresh

Collection name	Properties	Storage size	Documents	Avg. document size	Indexes	Total index size
breed_searches	-	36.86 kB	1	129.00 B	1	36.86 kB
pets	-	36.86 kB	1	355.00 B	1	36.86 kB
search_history	-	7.09 MB	2	1.76 MB	1	36.86 kB
user_settings	-	36.86 kB	1	122.00 B	1	36.86 kB

SwaggerUI :

Dog Breed Classifier API

0.1.0 OAS 3.1

/openapi.json

Authorize

pets

GET /api/pets Get Pets

POST /api/pets Add Pet

DELETE /api/pets/{pet_id} Delete Pet

POST /api/pets/{pet_id}/notes Add Note

DELETE /api/pets/{pet_id}/notes/{note_id} Delete Note

history

GET /api/history Get History

Frontend :

